

Trees

Root

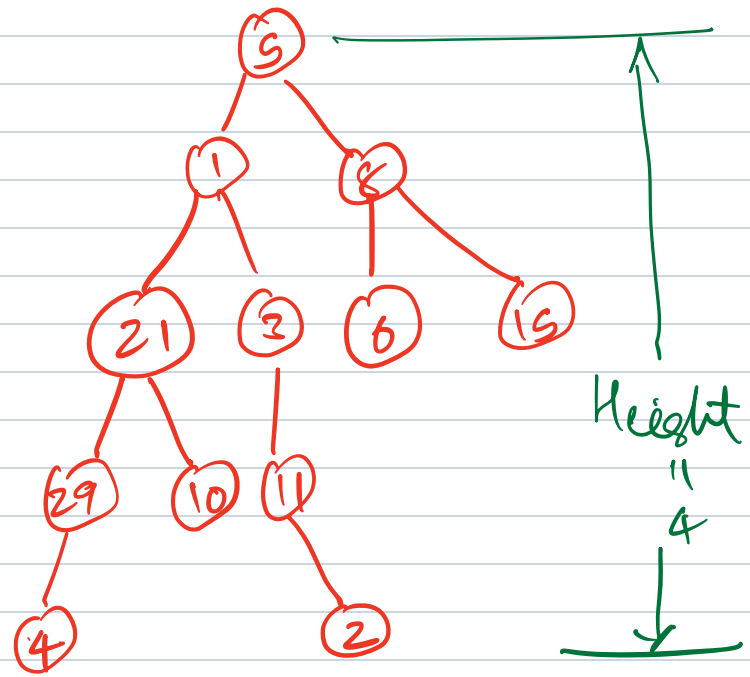
Parent, Child

Ancestor, Descendant

Sibling

Leaves, Internal Nodes

Depth, Height



This is a binary tree
because each node has
 ≤ 2 children.

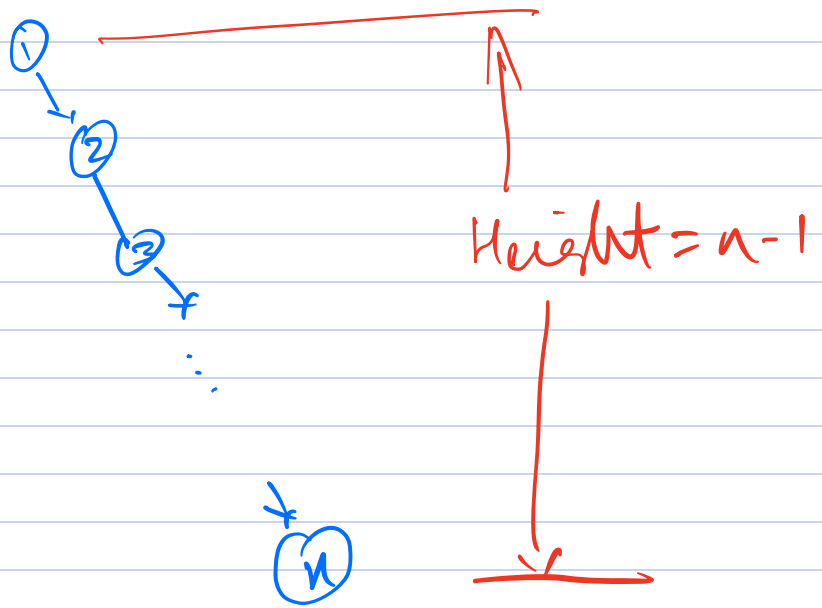
Leaves: Nodes that do not have children

Internal Nodes: Nodes that are not leaves.

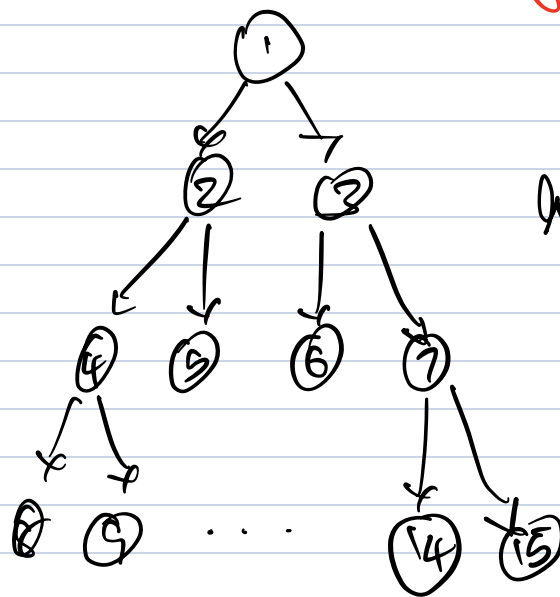
Depth (x): Depth of a node x is the no. of levels you have to go down before you reach x .

Height of the tree: Length of a longest root to leaf path.

Maximum Height of binary tree with n nodes



Minimum height of a binary tree with n nodes.



$$h \geq \lfloor \log_2 n \rfloor$$

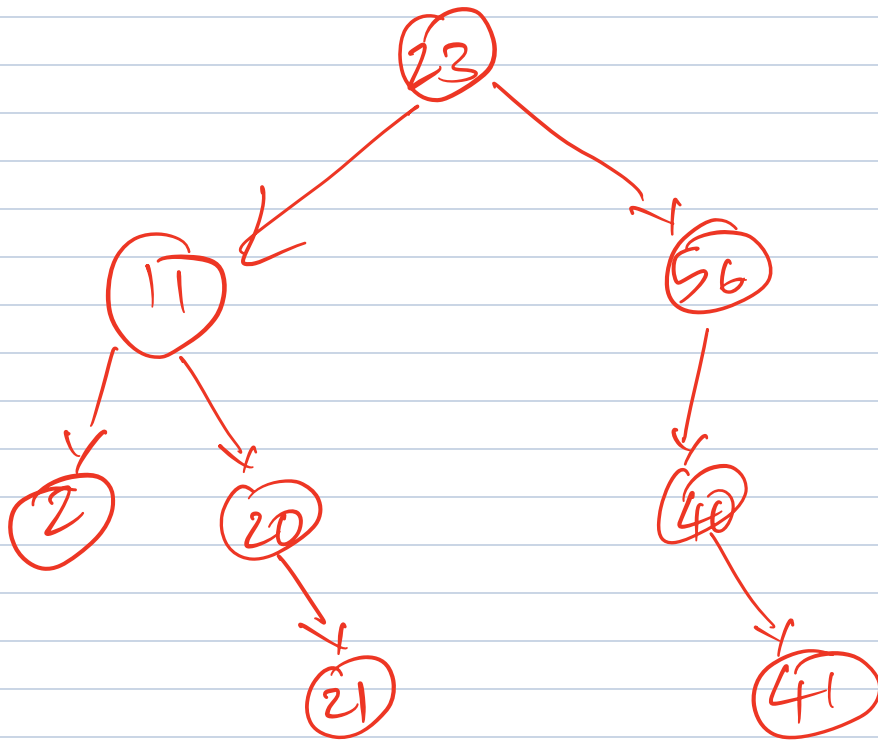
In general, height $h : \lfloor \log n \rfloor \leq h \leq n-1$

Alternatively $: h+1 \leq n \leq 2^{h+1} - 1$

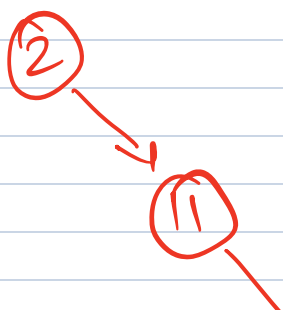
Insertion in Binary Search Tree (BST).

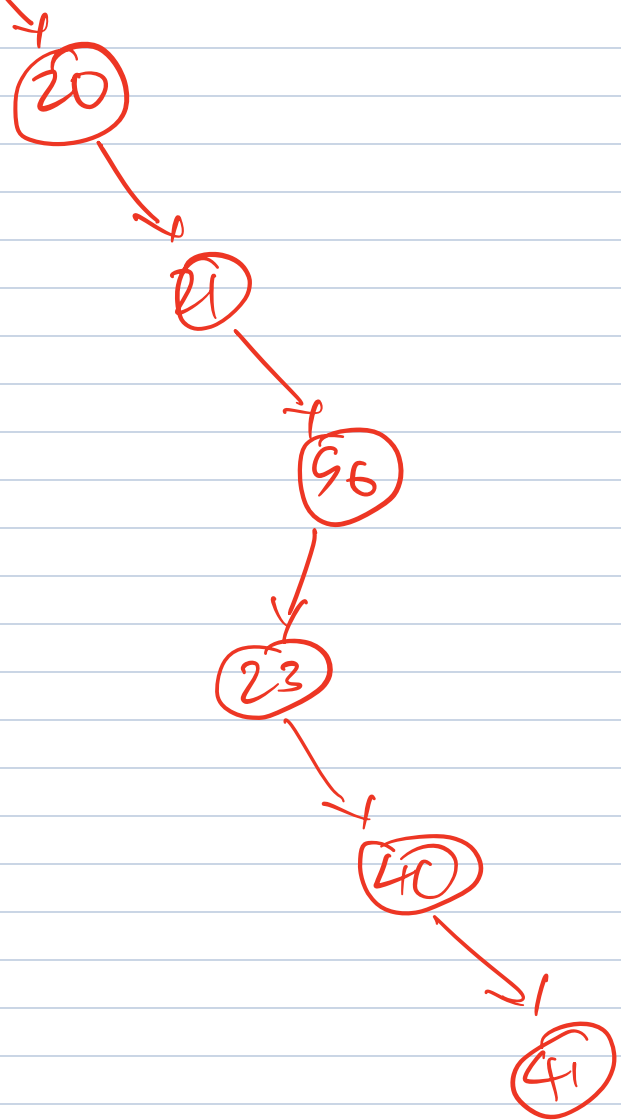
A: 23, 11, 20, 21, 2, 56, 40, 41.

↓
We insert in the given order.

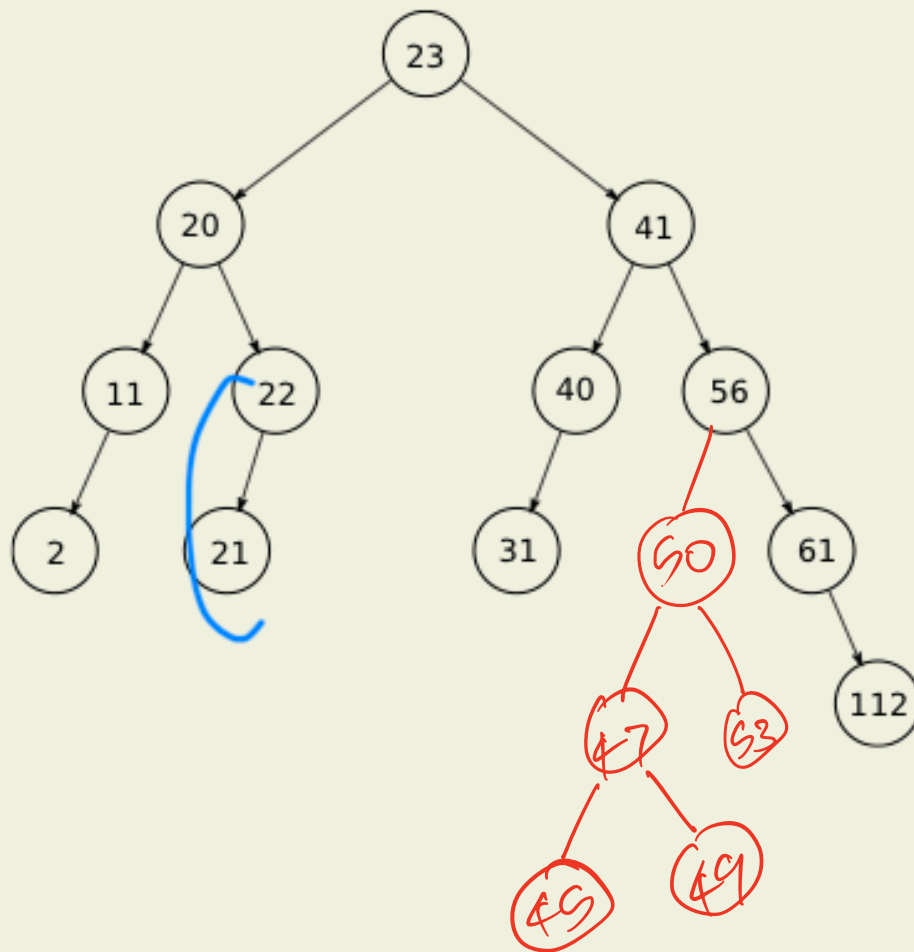


B: 2, 11, 20, 21, 56, 23, 40, 41





A binary tree is an ordered tree in which every node has at most children.

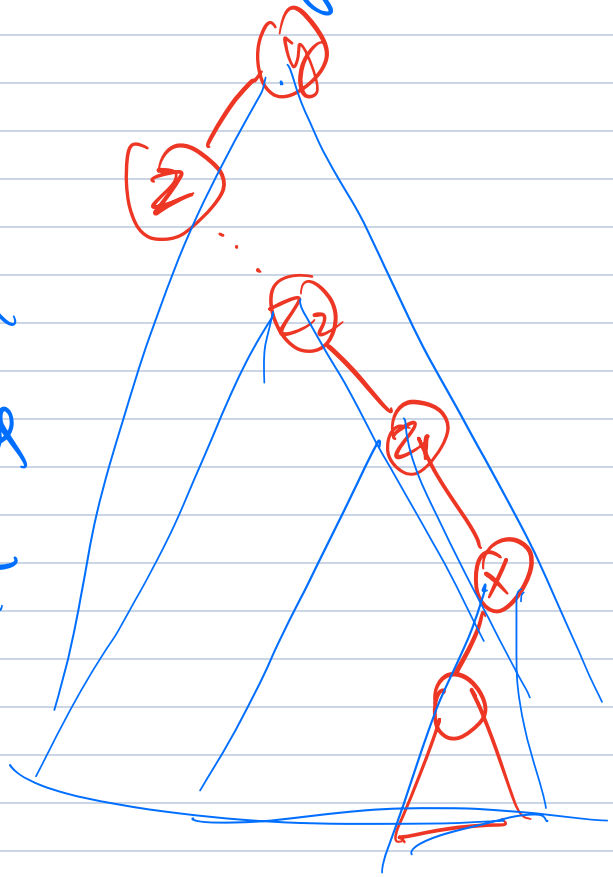


Where all can the successor be?

- ① Smallest element of right subtree (subtree rooted at the right child)
- ② Suppose there is no right child

of x , which is the node for which we are seeking the successor.

In this case, find the lowest ancestor y of x such that x is in the left subtree of y .



temp 1 = x

temp 2 = parent(x)

If temp 1 = left(temp 2)
then return temp 2.

Repeat

Else temp 1 = temp 2
temp 2 = parent(temp 1)

How do you compute the largest/smallest element of a BST?

d? ✓

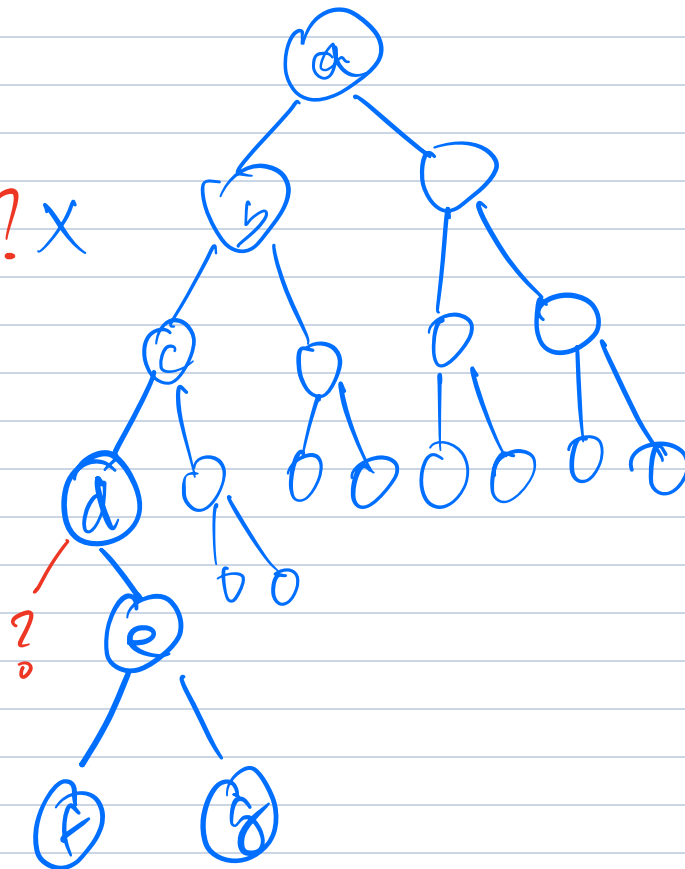
min(a, d)? x

e? x

f? x

g? x

c? x



c is ruled out since $d < c$

e is ruled out since $e > d$

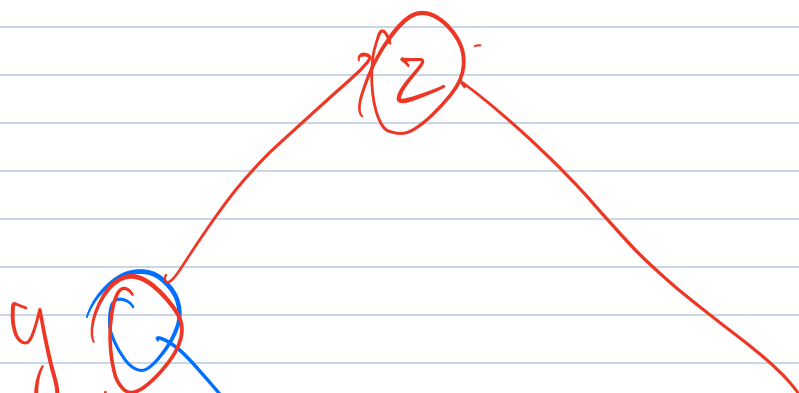
↳ So is f, g.

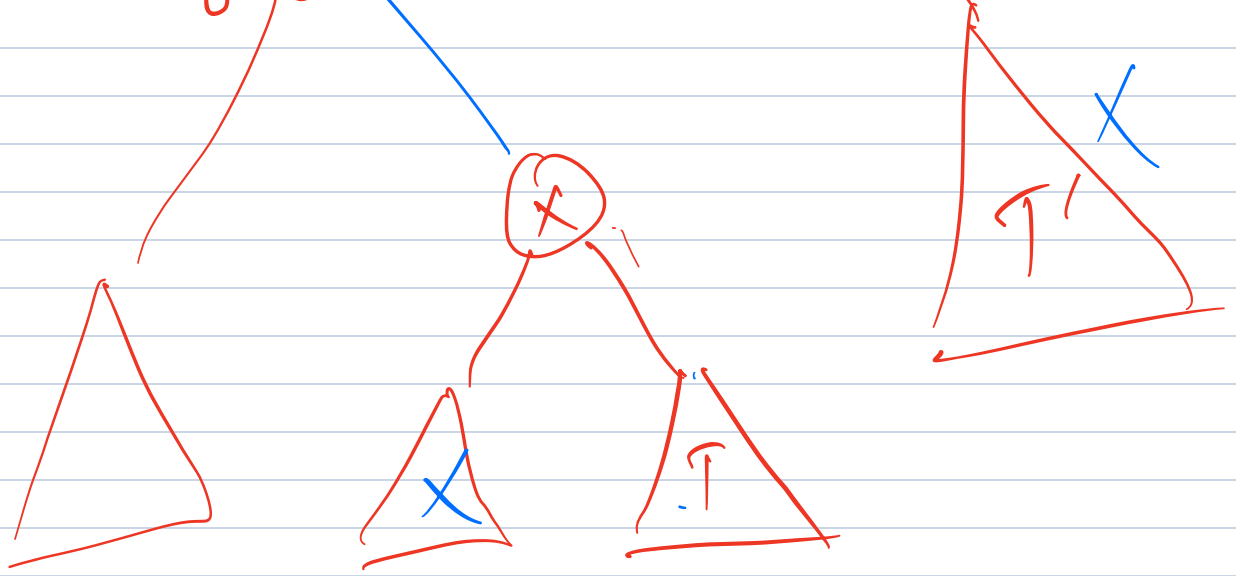
$\min(a, d) = d$ since d is in the left subtree of a .

To compute smallest element of a tree rooted at x , keep going left child repeatedly, till you reach a node y that does not have a left child.

Then y is the smallest value in the subtree rooted at x .

Same process with left/right swapped gives largest element.





Similarly, we can find the predecessor of a node x . Predecessor of x is the largest element that is smaller than x .

We can find the predecessor in the same way as the successor but swapping left and right.

In general, Insert, Search, Successor, Delete can take upto $O(h)$ time where h : Height of the BST.

We have already seen

$$\log n \leq h \leq n-1$$

So these tree operations can be $O(n)$ in the worst case.